Project plan: orthogonal factor lens for an AI equity basket

Project plan with detailed data download and cleaning plan. Prepared October 10, 2026.

Part 1: Overall roadmap

Phase 0: Design decisions (do before any code)

Write these down in one config or decision-log file, because every later step depends on them:

- **Research question:** which public market and macro drivers explain the returns and risk of an AI equity basket, and how do they split into non-redundant components?
- **AI basket definition:** about 25-40 names spread across the chain (chips, memory, networking, servers, hyperscalers, data-center operators, power generation, grid and electrical equipment, cooling, nuclear/uranium). Fix it with a written rule, not by hindsight.
- **Return frequency:** keep daily data, and decide later between daily, weekly, and monthly for modeling. Monthly is the natural frequency for the attribution report.
- **Estimation vs. evaluation split:** for example, in-sample 2016-2023 and out-of-sample 2024-2026, or an expanding window. Fix this early so factor selection never sees the evaluation period.
- **Success criteria:** stable betas, an interpretable factor set, a low condition number or VIF, and attribution that adds up exactly.

Phase 1: Data acquisition

Detailed in Part 2.

Phase 2: Data cleaning and QA

Detailed in Part 3.

Phase 3: Exploratory analysis and basket construction

- Build the basket's return series (equal-weight and cap-weight variants).
- Look at correlation structure, hierarchical clustering, and rolling correlations to SPY, SMH, and the utility and commodity ETFs.

- Sanity-check the AI → power link visually, e.g. rolling correlation of the basket to UTES/XLU/URA/GRID.

Phase 4: Candidate factor library

Build economically meaningful factors from the downloaded series, not just raw ETF returns:

- **Market:** SPY (or the excess return of the market).
- **Spread factors**, which are naturally less redundant: SMH − SPY (semiconductors), IGV − SPY (software), UTES − XLU (merchant power vs. regulated utilities), IWN − SPY (small-cap), the equal-weight vs. cap-weight spread if RSP is included.
- **Rates:** change in the 10-year yield, curve slope (10y − 3m), TIP-vs-IEF breakeven proxy.
- **Credit:** HYG − LQD or HYG − IEF.
- **USD:** UUP.
- **Commodities:** USO, UNG, COPX/copper, URA/URNM, GLD.
- **Volatility:** change in VIX.
- **Style:** low-vol (USMV − SPY), momentum if you include it.
- **Theme and capex:** GRID, PAVE, the AI-theme ETFs.

One caution: AI-theme ETFs (CHAT, WTAI, AIQ, etc.) hold the same stocks as your basket, so using them as factors is partly circular. Treat them as benchmarks rather than drivers, or at least flag the leakage.

Phase 5: Factor selection and orthogonalization

- **Selection:** correlation clustering to pick one representative per cluster, then VIF pruning, then penalized regression (LASSO or elastic net) and stability selection with time-ordered cross-validation.
- **Orthogonalization options to compare:**
  - Sequential Gram–Schmidt in an economically motivated order: market, then rates/USD/credit, then commodities, then sector and theme. The order matters and must be justified.
  - PCA on the factor set (orthogonal but harder to interpret).
  - Symmetric (Löwdin) orthogonalization, which stays closest to the original factors.
- Fit the orthogonalization on the estimation window only, then apply the same transformation out-of-sample. This avoids look-ahead.

Phase 6: Regression model

- Time-series regression of each stock and the basket on the orthogonal factors, with a full-sample and a rolling or EWMA version (e.g. 126-250 trading days).
- Report betas, R^2, adjusted R^2, and Newey–West t-stats. Because the factors are orthogonal, each beta can be estimated and read independently, so you can show that betas don't change when factors are added or removed (a nice demonstration of the orthogonality).
- Out-of-sample R^2 and beta stability.

Phase 7: Risk and P&L attribution

This is the "professional" part of the project.

- **P&L attribution:** period return = $\Sigma\, \beta_k \times \text{factor return}_k$ + alpha/residual. Use log returns or a linking method (Carino) so monthly numbers add up exactly across days.
- **Risk attribution:** variance = $w'B\Sigma_f B'w$ + specific variance. With orthogonal factors Σ_f is diagonal, so each factor's contribution is clean. Add marginal and component contributions to risk (Euler decomposition).
- **Monthly report, "what drove performance and risk this month":** a waterfall of factor contributions, a risk-share chart, and changes since last month.

Phase 8: Validation and robustness

Check how results change with different windows, frequencies, orthogonalization orders, the equal- vs. cap-weight basket, and with or without extreme days (e.g. March 2020). Check residual diagnostics and the stress-period behavior of the betas.

Phase 9: Deliverables

Report, final notebook(s), the monthly attribution report, the data dictionary, and a reproducible pipeline (config file, versioned data, fixed seeds).

Part 2: Data download plan

2.1 Principles

1. **Download once, keep the raw layer immutable.** Never edit raw files; all fixes happen in later layers.
2. **Save to Drive per batch**, not at the end. Colab sessions disconnect, so write each batch to Drive immediately and keep a manifest so the run can resume.

3. **Record a download timestamp.** Yahoo's adjusted prices are retroactively recalculated whenever a new dividend or split occurs, so the data is a snapshot.
4. **Download more than you think you need.** The whole dataset is small (about 600 tickers × ~2,700 days is tens of MB), so extra tickers cost almost nothing.

2.2 Date range, with two traps

- Start: 2016-01-01 (the first trading day is 2016-01-04).
- **yfinance's `end` is exclusive.** To include October 1, 2026, set `end = "2026-10-02"`. After downloading, explicitly assert that the last date is ⩽ 2026-10-01 and drop anything later.
- Use `auto_adjust=False` so you get Open, High, Low, Close, **Adj Close**, and Volume. You need both Close and Adj Close (see 3.4).
- Also download **dividends and splits** (`actions=True`, or from the ticker's history).
- Avoid `repair=True` in the raw download, because it silently modifies data. If you want repair, run it separately and compare.

2.3 Ticker universe, by tier

Tier	Contents	Source of the list
A	S&P 500 constituents (~503 tickers)	Wikipedia's S&P 500 table
B	Your `MACRO_ETFS` list and `EXTRA_STOCKS` (NRG, TLN)	Your lists
C	Non-S&P AI-chain names (see below)	Manual list
D	Index, rate, FX, volatility, and commodity series	Yahoo index and futures symbols
E	Additional ETFs for factor construction	Manual list

Tier A details

- Convert tickers for Yahoo: `BRK.B` → `BRK-B`, `BF.B` → `BF-B`.
- **Survivorship bias:** the current constituent list only contains survivors, and Yahoo does not carry most delisted tickers. Also scrape the historical changes table (additions/removals) from the same Wikipedia page and try downloading the removed tickers too; many will fail, and that is expected. Record both outcomes and note the limitation in the report. For an AI basket this is a modest issue, but it must be stated.
- Also store, per ticker, a flag for "in the S&P 500 today" and, where known, the date it joined.

Tier C: non-S&P names worth adding (verify each ticker exists on Yahoo and has history). Candidates by layer:

- Chips and memory: TSM, ASML, ARM, MRVL, and optionally 000660.KS (SK Hynix), 005930.KS (Samsung).
- Optical and networking: CRDO, ALAB, COHR, LITE, CLS.
- AI cloud and neoclouds: CRWV, NBIS, plus the miners-turned-hosts IREN, CIFR, APLD, CORZ, WULF.
- Power and nuclear: OKLO, SMR, NNE, BWXT, CCJ, LEU, UEC, BE, plus any of the independent power producers not already covered.
- Grid and electrical equipment: POWL, NVT, AMSC, and similar.

Many of these are recent IPOs, so their histories will be short. That is fine, but it makes the missing-data handling in Part 3 important.

Tier D: indexes, rates, FX, volatility, commodities

- Indexes: `^GSPC`, `^NDX`, `^RUT`, `^SOX`.
- Volatility: `^VIX`, and optionally `^VIX3M`.
- Rates: `^IRX` (13-week), `^FVX`, `^TNX`, `^TYX`. These are yields in percent, not prices.
- FX: `DX-Y.NYB` (dollar index).
- Futures: `CL=F`, `NG=F`, `HG=F`, `GC=F`.
- Crypto (relevant to miners and AI hosting): `BTC-USD`.

Tier E: extra ETFs worth considering (all cheap to include)

- Broad and style: QQQ, IWM, RSP, MTUM, QUAL, VLUE, SPHB, SPLV.
- Sectors: XLK, XLI, XLB, XLY, XLC, XLRE.
- Credit and rates: AGG, BIL, SHV, EMB, JNK.
- Energy transition and power: ICLN, TAN, NLR, XOP, OIH, XME.

Optional second source (macro): FRED series (10-year and 2-year yields, breakeven inflation, high-yield spread) and the Ken French daily factors (market, size, value, momentum, risk-free rate). They are free and public, and the French factors give you a standard benchmark and a clean risk-free series. Download them in the same session so everything is versioned together.

2.4 Checks on your ETF list

Several tickers in `MACRO_ETFS` are young (CHAT, THNQ, WTAI, IVES, IGPT, COPP, COPA, NUUR, SOXQ, and probably others), so they will have first dates well after 2016. A few (e.g. NUUR, COPA) should be verified to exist on Yahoo under exactly that symbol. This is

normal, but it constrains factor selection later: a factor with only 18 months of history cannot be part of a model estimated over 2016-2026. Record each series' **first valid date** in the manifest so this is visible.

2.5 Download mechanics

1. Process tickers in **batches of ~40-60**, with a short sleep between batches and retries (with backoff) on failure.
2. After each batch, write the result to Drive as Parquet, one file per batch or per ticker, in a dated folder such as `raw/yahoo_20261010/`.
3. Write a **manifest** (CSV) with one row per ticker: status (ok/empty/failed), error message, first date, last date, number of rows, download timestamp, batch id.
4. Re-run failed tickers once or twice in a second pass, individually, then freeze the list of permanent failures.
5. Download **metadata** once, separately: `quoteType`, sector, industry, currency, exchange, long name, and (for ETFs) category and inception date. This is slow and sometimes incomplete, so store whatever comes back and mark the rest as missing. Don't depend on Yahoo for fundamentals; its history there is short.
6. Save a copy of the **ticker lists as downloaded** (the S&P table with its scrape date) alongside the data.

2.6 Download QA (before cleaning)

- Count: expected tickers vs. received.
- Every series ends on 2026-10-01 (or its delisting date) and starts on 2016-01-04 or its listing date.
- SPY and `^GSPC` return series correlate at about 0.999 or better; if not, something is wrong.
- Spot-check 5-10 tickers against a second source (e.g. a known closing price on a known date).

Part 3: Data cleaning plan

3.0 Layers

`raw` (never edited) → `staged` (standardized types and schema) → `clean` (validated, flagged, analysis-ready) → `features` (returns, factors). Each layer is written to Drive with a version tag, so any step can be rerun without re-downloading.

3.1 Standardization (raw → staged)

- Flatten the MultiIndex columns into a long table (date, ticker, field) and a wide table per field (Adj Close, Close, Volume, etc.).
- Make the date index timezone-naive, sorted, and unique per ticker; assert no duplicates.
- Fix dtypes (float64 for prices to avoid rounding problems in returns; integers for volume).
- Tag each ticker as stock, ETF, index, rate, FX, or futures.

3.2 Trading calendar alignment

- Build a **master calendar** from the NYSE trading calendar (cross-checked with SPY's dates).
- For each ticker, separate three kinds of missing data:
  - **Leading NaNs** (before the listing date): legitimate; do nothing.
  - **Trailing NaNs** (after delisting or end of data): legitimate; do nothing.
  - **Internal gaps** (missing days between valid dates): suspicious; log every one.
- Do not forward-fill prices by default. Forward-filling creates artificial zero returns that bias volatility and betas downward. Preferred rule: leave a missing return as NaN, or compute the return across the gap as a multi-day return, and only fill short gaps (⩽ 1-3 days) if you decide it's needed, with a flag.
- Rows on non-trading days (some foreign or crypto series trade on weekends and holidays): drop them from the equity calendar. For BTC-USD, take the value on trading days only.

3.3 Invalid and implausible values

Flag (don't delete) each of these:

- Price ⩽ 0 or NaN (**exception: crude oil futures `CL=F` were negative on 2020-04-20, so log returns fail there; treat that day specially for futures and for roll-affected ETFs like USO**).
- High < Low, or Close outside [Low, High] (compare unadjusted Close with unadjusted High/Low only).
- Negative volume; zero volume on days with a price change.
- Stale prices: identical Close for many consecutive days with zero volume.

3.4 Corporate actions and adjustment check

This is the most error-prone part of Yahoo data.

- Compute your own total return as (Close_t + Dividend_t) / Close_{t-1} − 1 from the split-adjusted Close and the dividend table, and compare it with the return implied by Adj Close. Differences above a small tolerance indicate adjustment problems.
- The ratio Adj Close / Close should change only on ex-dividend dates. Unexplained steps are suspicious.
- Cross-check every day with |return| above ~35-40% against the splits table. A large one-day move without a split entry could be a missed split or a bad tick; a split entry with no move means it was already adjusted.
- Use **Adj Close for returns**. Use Close × Volume for dollar volume (Yahoo's volume is split-adjusted, but the unadjusted Close is not, so be careful and test it on a stock with a known split, e.g. NVDA).

3.5 Outlier detection

- Compute daily log returns and flag |r| above, say, 5 × a rolling MAD-based volatility or above 25%.
- **Bad-tick signature:** a spike followed by an almost exact reversal ($r_t \approx -r_{t+1}$). Flag as a likely data error.
- Classify each flagged return as: missed split, bad tick, genuine event (earnings, M&A, crash), or unknown. Genuine events stay in the data.
- Do not winsorize at the cleaning stage. Winsorizing is a modeling choice, made later and documented, and you'll want to test results with and without it.

3.6 Universe and quality filters

Create these as **flag columns in the metadata table** rather than deleting rows, so later decisions stay reversible:

- Minimum history (e.g. ⩾ 2 years, ⩾ 95% non-missing within its life).
- Minimum liquidity (e.g. median dollar volume above some threshold, plus price above $1).
- Share-class duplicates (GOOG/GOOGL, FOX/FOXA, NWS/NWSA): flag one so a basket doesn't double count.
- Ticker-change problems: Yahoo maps the full history to the current ticker (e.g. META), so old tickers from historical constituents may fail; log them.

3.7 Returns and non-price series

- **Equities/ETFs:** daily log and simple returns from Adj Close; weekly and monthly returns built by compounding daily returns, not by sampling prices at inconsistent dates.

- **Excess returns:** subtract the daily risk-free rate. `^IRX` is an annualized yield in percent, so convert (divide by 100, then by ~252); the French RF series is an alternative.
- **Yields (`^TNX`, `^IRX`, etc.):** these are levels, not prices. Use daily **changes in basis points**, not percentage returns. Likewise build the term spread as 10y − 3m.
- **VIX:** use changes (differences or log changes), and keep the level as a regime variable.
- **Futures and commodity ETFs:** continuous futures contain roll gaps, so returns near rolls can be artifacts. Prefer ETFs (USO, UNG, GLD) for factors, and treat the futures series as a cross-check only.
- **Non-U.S. tickers** (if you include Korean or Taiwanese names): they close at different times from U.S. markets, so same-day correlations are biased downward. Handle them with a lagged variable or exclude them from the core model.

3.8 Outputs of the cleaning stage

- `clean_adj_close` (wide), `clean_returns_daily`, `clean_volume`, `clean_dollar_volume`.
- `flags` table: one row per (date, ticker, issue type), with the decision taken.
- `universe_meta` table: ticker, type, sector, industry, inception or first valid date, last valid date, in S&P now, AI-basket membership, duplicate flag, quality flags.
- A **data quality report**: missingness heatmap, number of valid tickers over time, return histograms before/after, a list of all fixes and their counts.
- A short **data dictionary** and a `config` file with all thresholds (tolerances, outlier multipliers, liquidity cutoffs), so each choice is explicit and reproducible.

3.9 Cleaning QA checklist

1. Row counts before and after match the log of removals and fixes.
2. No NaN or infinite values in the returns matrix except legitimate ones (leading/trailing, flagged gaps).
3. SPY vs `^GSPC` return correlation and tracking error are sensible.
4. Sector ETF returns are consistent with the sum of their large constituents (rough check).
5. Re-running the cleaning pipeline from raw gives byte-identical output (determinism).
6. Everything is uploaded to Drive with the version tag and a checksum.